

Final Implementation Plan

Final Implementation Plan

OpenClaw + Stratus: Counterfactual Guardrail for Telehealth Scheduling Surges

Track

Stratus X1 Hackathon

Track: `Multi-Agent Systems`

1. Final Objective

We are building a multi-agent guardrail for telehealth scheduling incidents caused by public-health surges such as COVID, flu, RSV, or sudden specialist-slot releases.

The product goal is **not** to mathematically optimize the entire healthcare access system. The simulator is only a realistic operational sandbox. The actual product goal is:

- detect a real-looking access incident
- reject the dangerous local reflex
- plan the safest next one to three actions
- execute only what is needed
- verify recovery over time
- beat a naive baseline on both quality and speed

Product claim:

> OpenClaw gives the system hands. Stratus gives it foresight. Together they form a counterfactual remediation guardrail for telehealth scheduling operations.

2. Final Story

At 9:00 AM, a health system opens a limited release of telehealth and same-week specialist appointments during a surge event.

Patient demand spikes. The patient portal remains reachable, but the scheduling service depends on eligibility / benefits verification, and that dependency becomes slow or flaky. Patients retry. The system retries. Slot holds remain open while booking is unresolved. What begins as a mild dependency issue becomes an access incident:

- scheduling latency rises
- retries amplify load
- slot holds clog limited capacity
- abandonment rises
- booking completion falls
- fairness degrades because some users repeatedly occupy scarce slots while others never get through

The obvious reflex is to restart the degraded dependency. That reflex is intuitive but dangerous. During a retry storm, restarting the dependency can worsen backlog pressure and extend recovery.

The demo must show that:

1. the incident appears to happen naturally
2. OpenClaw wakes up from the background
3. Stratus rejects the dangerous reflex
4. the system executes a safer action or short guarded sequence
5. verification proves whether recovery happened
6. the result is written back to a compact case library for future retrieval

3. Non-Negotiable Final-Demo Principles

1. **One startup prompt**

- OpenClaw is armed once and stays in the same task.
 - It waits for incidents in the background and wakes automatically.
2. ****Business-event trigger****
 - Do not surface a fake `Trigger Incident` button in the main story.
 - The UI should expose business triggers such as `Open Flu Surge Telehealth Window`.
 3. ****Single-action flow must be stable****
 - Multi-step is an extension over a solid one-action workflow.
 - Step 1 must work every time.
 4. ****Multi-step is visible even if only step 1 executes****
 - The plan must display `stabilize -> recover -> normalize`.
 5. ****Comparison must include time****
 - Do not compare only final effect.
 - Compare speed to first action, speed to safe state, and action efficiency over time.

4. Reference Stack

- `Python + FastAPI` for local services and orchestration glue
- `OpenClaw` for browser execution and agent orchestration
- `Stratus` for shortlist ranking and guarded sequence planning
- `Prometheus + Alertmanager` for alerts and verification
- `SimPy` for discrete-event simulation
- `DuckDB + Parquet` for compact case summaries + raw traces
- `SQLite` is acceptable instead of DuckDB if needed, but DuckDB is preferred for analytics

5. Core System Architecture

5.1 Surfaces

- `/operations`
 - business-facing telehealth scheduling stability board
- `/`
 - Guardrail Console with Incident / Decision / Execution / Verdict views
- `/openclaw-execution`
 - dedicated browser surface for OpenClaw
- `/feature-flags`
 - manual fallback controls only

5.2 Background Activation

- Alertmanager writes the latest incident payload to `alerts/latest.json`
- OpenClaw is armed with:
 - `.venv/bin/python run.py alerts/latest.json --phase await-demo`
- The watcher:
 - observes the alert file
 - latches `pending\_incident`
 - acknowledges it
 - prepares demo artifacts
- OpenClaw then continues immediately through browser execution, verify or fallback, summary, and returns to `await-demo`

5.3 Multi-Agent Roles

- `Sentinel Agents`
 - parallel observation
- `Case Retrieval Agent`
 - retrieves similar abstracted cases
- `Guardrail Planner Agent`
 - shortlist + 1-3 step guarded sequence
- `Operator Agent`
 - OpenClaw execution

- `Evaluation Agent`
 - verify, compare baseline, write back compact case summary

6. Simulation Design

6.1 Modeling Philosophy

Do **not** use a black-box ML simulator as the main engine.

Use:

- discrete-event simulation
- queueing dynamics
- explicit retry amplification
- finite slot-hold inventory
- light stochasticity for robustness

That makes the simulator causal, interpretable, and suitable for counterfactual planning.

6.2 Base Dynamics

The simulator should use the following base layer:

- arrivals: `non-homogeneous Poisson process`
- service time: `log-normal`
- queue: simplified `M/G/k`
- retry amplification: explicit retry multiplier or retrial queue
- abandonment: hazard increases with wait time
- slot holds: finite-capacity inventory pool with TTL

6.3 Important Modeling Correction

Actions do **not** only change the Poisson arrival process.

Instead, actions can change:

- `lambda\_ext(t)` external arrivals
- `lambda\_retry(t)` retry-generated arrivals
- `lambda\_gated(t)` traffic sent to manual fallback / callback
- `mu\_service(t)` service speed
- `k(t)` concurrency or capacity
- `ttl\_hold(t)` slot hold duration
- `r\_region(t)` traffic-routing split
- `h\_abandon(t)` abandonment hazard

Use:

$\lambda_{\text{eff}}(t) = \lambda_{\text{ext}}(t) + \lambda_{\text{retry}}(t) + \lambda_{\text{reroute_in}}(t) - \lambda_{\text{gated}}(t)$

6.4 Randomness / Robustness

Use small controlled noise, not unstable randomness:

- $\lambda_{\text{ext}}(t) = \lambda_{\text{base}}(t) * \exp(\epsilon_t)$
- ϵ_t can be Gaussian or AR(1) noise with small variance
- keep shared random seeds across methods for fair comparison
- use common random numbers so baseline and guardrail see the same underlying demand trace

6.5 Checkpoint Sampling

Sample state every `5s` for observability.

Decision checkpoints:

- `T+15s`
- `T+30s`
- `T+60s`

Checkpoint outcome labels:

- `strong\_recovery`
- `partial\_recovery`

- ``severe_failure``

Behavior:

- ``strong_recovery`` -> stop
- ``partial_recovery`` -> execute step 2 if available
- ``severe_failure`` -> replan with Stratus

7. Scenario Portfolio

The final demo should support four realistic scenarios.

Scenario A: Surge Retry Spiral

- business trigger: ``Open Flu Surge Telehealth Window``
- mechanics:
 - external demand spike
 - eligibility slows, but is not fully down
 - scheduling retries amplify
 - abandonment rises
- dangerous reflex:
 - ``restart_eligibility_service``
- likely safer actions:
 - ``throttle_booking_retries``
 - ``enable_eligibility_circuit_breaker``
 - ``increase_retry_backoff``
- guardrail rule:
 - if retry amplification is dominant and the action is still early, ``throttle_booking_retries`` should win
 - ``enable_eligibility_circuit_breaker`` should win only when dependency health is the dominant hidden bottleneck

Scenario B: Fairness Breakdown / Slot Hold Clog

- business trigger: ``Release High-Demand Specialist Slots``
- mechanics:
 - repeated attempts hold scarce slots
 - hold TTL is too long
 - access fairness collapses
- dangerous reflex:
 - apply a generic dependency fix or traffic move without clearing trapped slot inventory
- likely safer actions:
 - ``shorten_slot_hold_ttl``
 - ``route_to_callback_queue``
 - ``reserve_priority_slots``
- guardrail rule:
 - ``enable_eligibility_circuit_breaker`` must not rank first here unless evidence shows dependency degradation truly dominates
 - one of the inventory / fairness actions must rank above the generic dependency fix
 - if staffed callback capacity is still available, ``route_to_callback_queue`` can outrank TTL-only relief as the first move

Scenario C: Eligibility Verification Flap

- business trigger: ``Open Insurance-Verified Same-Day Visits``
- mechanics:
 - dependency errors intermittently
 - retry pressure moderate
 - booking completion drops
- dangerous reflex:
 - ``restart_eligibility_service`` before backlog is controlled

- likely safer actions:
 - ``enable_eligibility_circuit_breaker``
 - ``increase_retry_backoff``

Scenario D: Regional Access Saturation

- business trigger: ``Divert Regional Overflow to Primary Portal``
- mechanics:
 - primary region saturates
 - secondary region has limited headroom
- dangerous reflex:
 - full failover
- likely safer actions:
 - ``shift_scheduling_traffic_10_percent``
 - ``pause_low_priority_intake``
 - ``route_to_callback_queue``
- guardrail rule:
 - traffic shift is safe only if projected post-shift ``secondary_region_headroom > threshold``
 - even when projected headroom is safe, traffic shift should usually be a step-2 recovery move unless retry pressure and queue depth are already contained
 - if projected headroom is below threshold, ``route_to_callback_queue`` or retry shaping should outrank traffic shift

8. Action Library

Base library:

- ``throttle_booking_retries``
- ``enable_eligibility_circuit_breaker``
- ``increase_retry_backoff``
- ``shorten_slot_hold_ttl``
- ``route_to_callback_queue``
- ``reserve_priority_slots``
- ``shift_scheduling_traffic_10_percent``
- ``shift_scheduling_traffic_25_percent``
- ``pause_low_priority_intake``
- ``enable_bot_challenge``
- ``disable_online_scheduling``
- ``restart_eligibility_service``

Each action must define:

- ``id``
- ``label``
- ``category``
- ``description``
- ``expected_parameter_delta``
- ``execution_surface``
- ``business_cost``
- ``reversibility``
- ``unsafe_if``

8.1 Action Definition Contract

```
```json
{
 "id": "enable_eligibility_circuit_breaker",
 "label": "Enable Eligibility Circuit Breaker",
 "category": "retry_overload_control",
 "description": "Fail fast on degraded eligibility checks to break retry amplification.",
 "execution_surface": "browser",
```

```

"business_cost": "low",
"reversibility": "high",
"expected_parameter_delta": {
 "lambda_retry_multiplier": -0.65,
 "mu_eligibility_effective": 0.0,
 "h_abandon": -0.20
},
"unsafe_if": [
 "eligibility dependency fully healthy",
 "error budget already exhausted by fail-fast policy"
]
}
...

```

## 9. Shortlist Mechanism

Start from the full action library and reduce to `4-6` actions.

### 9.1 Shortlist Inputs

- current incident state
- scenario family
- hard constraints
- retrieved similar cases
- action library

### 9.2 Shortlist Scoring Factors

- symptom relevance
- scenario compatibility
- fairness compatibility
- regional compatibility
- historical usefulness from case library
- estimated blast radius
- timing sensitivity
- hidden constraint compatibility

### 9.3 Tony's Required Shortlist Logic

Tony's shortlist mechanism must explicitly account for:

- `retry\_death\_spiral`
  - `throttle\_booking\_retries` outranks circuit breaker when retry pressure is dominant and the action is early enough to matter
- `payment\_gateway\_flap`
  - `enable\_eligibility\_circuit\_breaker` outranks throttle because dependency degradation is dominant
- `seat\_hold\_clog`
  - at least two of `shorten\_slot\_hold\_ttl`, `route\_to\_callback\_queue`, `reserve\_priority\_slots` must enter the shortlist
  - `enable\_eligibility\_circuit\_breaker` must be penalized unless hidden evidence says dependency health is the primary cause
  - if callback capacity is available, `route\_to\_callback\_queue` can outrank TTL-only relief as the first move
- `regional\_saturation`
  - `shift\_scheduling\_traffic\_10\_percent` only enters as a top first-step action when projected post-shift secondary headroom stays above threshold and retry pressure is already contained
  - otherwise diversion / callback actions or retry shaping must outrank it
- `restart\_eligibility\_service`
  - keep it visible as the tempting local reflex
  - penalize it unless queue depth and retry share are already below stabilization thresholds

## 9.4 Shortlist Output Contract

```
```json
{
  "shortlist_id": "uuid",
  "scenario_family": "surge_retry_spiral",
  "scoring_version": "v1",
  "selected_actions": [
    "enable_eligibility_circuit_breaker",
    "throttle_booking_retries",
    "increase_retry_backoff",
    "shift_scheduling_traffic_10_percent"
  ],
  "selection_rationale": [
    "eligibility degradation is primary bottleneck",
    "retry amplification is high",
    "secondary region headroom is limited"
  ]
}
```
```

## 10. Sequence Planning

Sequence planning is Tony's core innovation area.

### 10.1 Rule

Plan `1-3` actions.

- always produce a sequence artifact
- only execute step 2 or step 3 if needed
- replan only after a failed checkpoint

### 10.2 Sequence Template

- `Step 1: Stabilize`
- `Step 2: Recover`
- `Step 3: Normalize`

### 10.3 Tony's Sequence Rules

Tony's sequence planner must follow these rules:

- `retry\_death\_spiral`
  - `Step 1` should usually be `throttle\_booking\_retries`
  - `Step 2` can be `enable\_eligibility\_circuit\_breaker` only if dependency health remains the dominant blocker at checkpoint
- `payment\_gateway\_flap`
  - `Step 1` should usually be `enable\_eligibility\_circuit\_breaker`
  - `Step 2` can be `increase\_retry\_backoff`
- `seat\_hold\_clog`
  - `Step 1` should be one of the inventory / fairness actions
  - `Step 2` should add diversion or fairness protection if slot utilization remains high
- `regional\_saturation`
  - `Step 1` should usually reduce retry pressure or divert overflow without consuming secondary-region margin
  - `Step 2` can apply cautious traffic shift once projected post-shift secondary headroom remains above threshold and retry pressure is already reduced
- `restart\_eligibility\_service`
  - can appear only as a later recovery step
  - must never be step 1 while retry share or queue depth is above threshold
- replan only after a checkpoint returns `partial\_recovery` or `severe\_failure`

## 10.4 Sequence Output Contract

```
```json
{
  "plan_id": "uuid",
  "shortlist_id": "uuid",
  "steps": [
    {
      "step": 1,
      "action_id": "enable_eligibility_circuit_breaker",
      "objective": "stabilize",
      "why_now": "immediately breaks retry amplification",
      "expected_parameter_delta": {
        "lambda_retry_multiplier": -0.65
      },
      "expected_metric_delta": {
        "retry_amplification_factor": "down",
        "scheduling_latency_p95_ms": "down"
      },
      "stop_if": "risk_level == low",
      "continue_if": "partial_recovery",
      "replan_if": "severe_failure"
    },
    {
      "step": 2,
      "action_id": "increase_retry_backoff",
      "objective": "recover",
      "why_now": "smooths residual pressure after stabilization",
      "stop_if": "latency_p95_ms < 1200",
      "continue_if": "partial_recovery",
      "replan_if": "blast_radius rises"
    }
  ],
  "checkpoints_sec": [15, 30, 60]
}
```
```

## 11. Observability

Tony and Hansen must make action effects visible over time.

### 11.1 Prometheus Metrics

Technical:

- `scheduling\_latency\_ms\_bucket`
- `eligibility\_latency\_ms\_bucket`
- `scheduling\_error\_rate`
- `booking\_retry\_rate`
- `retry\_amplification\_factor`
- `scheduling\_queue\_depth`
- `eligibility\_utilization`

Business:

- `booking\_completion\_rate`
- `scheduling\_abandonment\_rate`
- `slot\_hold\_utilization`
- `slot\_hold\_expiration\_rate`
- `manual\_callback\_queue\_depth`



- `priority\_slot\_fill\_rate`

Fairness:

- `cohort\_wait\_gap\_sec`
- `hold\_concentration\_hhi`
- `duplicate\_request\_share`

Recovery:

- `time\_to\_first\_action\_sec`
- `time\_to\_low\_risk\_state\_sec`
- `actions\_taken\_total`
- `replans\_total`

## 11.2 What the Final Demo Should Show

Do not show every metric.

Recommended visible metrics:

- `scheduling\_latency\_p95\_ms`
- `retry\_amplification\_factor`
- `booking\_completion\_rate`
- `scheduling\_abandonment\_rate`
- `slot\_hold\_utilization`
- one fairness metric, preferably `cohort\_wait\_gap\_sec`
- `time\_to\_low\_risk\_state\_sec`
- baseline vs guardrail first action

## 12. Triggers

The main story should use business-event triggers.

### 12.1 Trigger Contract

```
```json
{
  "trigger_id": "open_flu_window",
  "label": "Open Flu Surge Telehealth Window",
  "scenario_id": "surge_retry_spiral",
  "alert_profile": {
    "alertname": "SchedulingRetrySpiral",
    "service": "scheduling",
    "dependency": "eligibility",
    "summary": "Scheduling retry pressure above safe baseline"
  }
}
```
```

### 12.2 UI Buttons

Expose:

- `Open Flu Surge Telehealth Window`
- `Release High-Demand Specialist Slots`
- `Open Insurance-Verified Same-Day Visits`
- `Divert Regional Overflow to Primary Portal`

## 13. Prompt Contract

### 13.1 Startup Prompt

```
```text
```

Use the incident_guardrail skill in this workspace and stay in the same task until each incident workflow is complete. Start by running ``.venv/bin/python run.py alerts/latest.json --phase await-demo``, let that command block until the next incident is latched and the demo artifacts are

prepared, then continue immediately through browser execution, verification or fallback, summary, and return to await-demo mode. Do not stop after `await-demo` returns.

13.2 Resume Prompt

```text

Use the incident\_guardrail skill in this workspace. An incident is already acknowledged and the demo artifacts are prepared. Resume now by reading `outputs/alert\_latest\_openclaw\_demo.json` and `outputs/alert\_latest\_browser\_playbook.json`, executing the browser step in `/openclaw-execution`, then running verify or fallback. If fallback is used, still open `/openclaw-execution?stage=verdict` before summarizing the verdict and returning to await-demo mode.

```

14. Case Library

14.1 Goal

The case library must support:

- retrieval for shortlist and planning
- post-run writeback
- baseline comparison
- compact context usage

14.2 Storage Layout

- `cases/raw/\*.parquet`
 - full time series and checkpoints
- `cases/index.duckdb`
 - queryable compact summaries
- `cases/summaries/\*.json`
 - lightweight portable summaries

14.3 Two-Layer Writeback

Never write full raw traces into prompt-facing memory.

Write:

1. `raw\_trace.parquet`
2. `case\_summary.json`

14.4 Summary Contract

```json

```
{
 "case_id": "uuid",
 "scenario_family": "surge_retry_spiral",
 "seed": 42,
 "pre_features": {
 "latency_p95_ms": 2300,
 "retry_factor": 4.1,
 "abandonment_rate": 0.29,
 "slot_hold_utilization": 0.91,
 "regional_headroom": 0.18
 },
 "retrieved_case_ids": ["case_001", "case_017"],
 "shortlist": [
 "enable_eligibility_circuit_breaker",
 "throttle_booking_retries",
 "increase_retry_backoff"
],
}
```

```

"planned_sequence": [
 "enable_eligibility_circuit_breaker",
 "increase_retry_backoff"
],
"executed_steps": [
 "enable_eligibility_circuit_breaker"
],
"checkpoint_decisions": [
 {"t_sec": 15, "status": "partial_recovery", "decision": "continue"},
 {"t_sec": 30, "status": "strong_recovery", "decision": "stop"}
],
"guardrail_outcome": {
 "time_to_low_risk_state_sec": 35,
 "blast_radius": "low",
 "fairness_score": 0.86,
 "actions_taken": 1,
 "replans": 0
},
"baseline_outcome": {
 "first_action": "restart_eligibility_service",
 "time_to_low_risk_state_sec": 120,
 "actions_taken": 2,
 "replans": 1
}
}
...

```

## 14.5 Retrieval Policy

Use only compact summaries for online retrieval.

Similarity should use:

- exact or near-exact scenario family
- normalized pre-incident feature vector
- optional action-family overlap

Retrieve at most `3` similar cases into prompt context.

## 15. Comparison Pipeline

Eason owns this.

### 15.1 Comparison Principles

- baseline and guardrail must use the same seed
- compare over time, not just final state
- run in parallel when possible

### 15.2 Required Metrics

- `baseline\_first\_action`
- `guardrail\_first\_action`
- `time\_to\_first\_action\_sec`
- `time\_to\_low\_risk\_state\_sec`
- `latency\_auc`
- `abandonment\_auc`
- `fairness\_auc`
- `actions\_taken`
- `replans`
- `final\_risk\_level`
- `decision\_fidelity`

## 15.3 Comparison Output Contract

```
```json
{
  "comparison_id": "uuid",
  "scenario_family": "surge_retry_spiral",
  "shared_seed": 42,
  "baseline": {
    "first_action": "restart_eligibility_service",
    "time_to_first_action_sec": 2,
    "time_to_low_risk_state_sec": 120,
    "latency_auc": 81200,
    "abandonment_auc": 14.6,
    "actions_taken": 2,
    "replans": 1
  },
  "guardrail": {
    "first_action": "enable_eligibility_circuit_breaker",
    "time_to_first_action_sec": 3,
    "time_to_low_risk_state_sec": 35,
    "latency_auc": 42100,
    "abandonment_auc": 6.3,
    "actions_taken": 1,
    "replans": 0
  },
  "winner": {
    "speed": "guardrail",
    "fairness": "guardrail",
    "efficiency": "guardrail",
    "final_verdict": "guardrail"
  }
}
```
```

## 16. Final Communication Contracts

### 16.1 Incident State

```
```json
{
  "event_id": "uuid",
  "scenario_id": "surge_retry_spiral",
  "seed": 42,
  "detected_at_utc": "timestamp",
  "metrics": {},
  "state_params": {},
  "constraints": {},
  "retrieved_case_ids": []
}
```
```

### 16.2 Checkpoint Log

Use JSONL:

```
```json
{"t_sec":0,"type":"before","metrics":{},"params":{}}
{"t_sec":15,"type":"checkpoint","step":1,"decision":"continue","metrics":{},"params":{}}
{"t_sec":30,"type":"checkpoint","step":1,"decision":"stop","metrics":{},"params":{}}
```
```

```
{"t_sec":35,"type":"final","status":"low_risk_state","metrics":{}}
...
```

### 16.3 Final Verdict

Must always include:

- incident summary
- dangerous reflex rejected
- chosen safer action
- whether fallback was used
- predicted vs actual
- time to low-risk state
- baseline vs guardrail

## 17. Responsibilities

Responsibilities below are intentionally specific enough that an AI agent with no extra context should be able to implement them.

### Hansen

#### Primary ownership:

- OpenClaw wake / wait / resume loop
- Guardrail Console shell
- business trigger surface
- final verdict surface
- cross-module contracts

Implement:

1. Update `/operations` so the visible triggers are business events, not raw incident buttons.
2. Keep `/openclaw-execution` as the single browser surface.
3. Make sure fallback always returns the browser to `/openclaw-execution?stage=verdict`.
4. Add sequence visibility to the UI:
  - show planned step 1/2/3
  - show current checkpoint
  - show whether step 2 is needed
5. Add final comparison cards:
  - baseline first action
  - guardrail first action
  - time to low-risk state
  - actions taken
6. Keep prompt docs and `SKILL.md` aligned with actual repo behavior.
7. Lock schemas in `docs/` and ensure all teams code to those schemas.

Files Hansen should own:

- `tools/visual\_control\_plane.py`
- `docs/openclaw\_demo\_prompt.md`
- `docs/final\_demo\_script.md`
- integration parts of `run.py`

Hansen acceptance tests:

- one startup prompt arms OpenClaw
- one business event triggers the incident
- browser or fallback always lands on final verdict
- all four views stay coherent

### Charlie

#### Primary ownership:

- scenario realism
- simulation parameter realism

- fairness / access definitions
- demo-facing scenario story

Implement:

1. Create scenario configs for all four scenarios with:
  - baseline external arrival curve
  - service distributions
  - queue capacity
  - retry multipliers
  - slot hold TTL and capacity
  - fairness-sensitive metrics
2. Define which metrics matter most for each scenario.
3. Specify dangerous reflex and acceptable safer actions per scenario.
4. Define business triggers and UI labels.
5. Define the fairness story:
  - what counts as unfairness
  - how it is measured
  - what actions improve or worsen it
6. Work with Hansen to make the visible board feel like real telehealth access ops.

Files Charlie should own:

- `scenarios/\*.json`
- `tools/scenario\_catalog.py`
- scenario docs under `docs/`

Charlie acceptance tests:

- scenarios are believable
- fairness metric changes match the story
- the demo can be narrated without hand-waving

## Tony

### Primary ownership:

- shortlist generation
- guarded sequence planning
- checkpoint logic
- replan behavior
- action-effect observability

Implement:

1. Build the shortlist scorer from the full library to `4-6` actions.
2. Build `sequence\_plan.json` output with `1-3` steps.
3. Add explicit per-action parameter deltas:
  - retry multiplier
  - routing fraction
  - service speed
  - hold TTL
  - gating to callback/manual fallback
4. At every checkpoint, compute:
  - current status
  - continue / stop / replan decision
5. Log checkpoint reasoning and action effects for case writeback.
6. If step 1 fails partially, decide whether to:
  - execute planned step 2
  - or replan with Stratus
7. Keep sequence planning compatible with current one-action stable flow.
8. Implement scenario-sensitive threshold rules:
  - `retry\_death\_spiral`: early throttle beats circuit breaker unless dependency health is the

dominant blocker

- ``seat_hold_clog``: inventory / fairness actions outrank generic dependency actions, with callback diversion preferred when staffed overflow capacity is still available
- ``regional_saturation``: traffic shift is permitted only when projected post-shift headroom clears threshold and should usually be a step-2 move after stabilization
- ``restart_eligibility_service``: only promote it after stabilization

9. Log how each executed step changed:

- retry multiplier
- effective service rate
- slot-hold pressure
- callback diversion
- secondary-region headroom

10. Make checkpoint output readable enough that Hansen can show it directly in the console and verdict UI.

Files Tony should own:

- ``tools/stratus_guardrail.py``
- new ``agents/sequence_planner.py`` if needed
- shortlist logic in ``agents/candidate_actions.py`` or a dedicated scorer module

Tony acceptance tests:

- every scenario produces a sensible shortlist
- every scenario produces a valid sequence plan
- checkpoints are logged
- replan only occurs on defined failure conditions
- ``retry_death_spiral``: throttle ranks above circuit breaker when retry dominance is primary
- ``payment_gateway_flap``: circuit breaker ranks above throttle
- ``seat_hold_clog``: slot-hold/fairness action or callback diversion ranks above circuit breaker
- ``regional_saturation``: shift is penalized whenever projected post-shift headroom is below threshold or retry pressure is still too high for a first-step move
- ``restart_eligibility_service`` never appears as step 1 before stabilization

## Eason

### Primary ownership:

- case-library retrieval
- baseline policy
- comparison engine
- final performance stats over time

Implement:

1. Build retrieval over compact case summaries only.
2. Run baseline and guardrail on the same seed and demand trace.
3. Compare:
  - first action
  - time to first action
  - time to low-risk state
  - latency / abandonment / fairness over time
  - actions taken
  - replans
4. Write both:
  - raw trace
  - compact case summary
5. Design summary abstraction so prompt context stays small:
  - pre-features
  - shortlist
  - planned sequence

- executed steps
- checkpoint decisions
- final compact outcome

6. Expose baseline-vs-guardrail metrics in a single final artifact for Hansen to render.

Files Eason should own:

- `agents/baseline.py`
- `agents/evaluator.py`
- `tools/case\_library.py`
- `run\_baseline.py` if kept

Eason acceptance tests:

- retrieval works without loading full raw traces into prompt context
- baseline and guardrail compare on the same seed
- final comparison is time-based, not only single-point

## 18. Implementation Order

1. Lock schemas in code and docs.
2. Charlie finalizes scenario configs and realism tables.
3. Tony implements shortlist + sequence + checkpoint logic.
4. Eason implements retrieval + baseline + comparison over time.
5. Hansen integrates the surfaces and final verdict cards.
6. End-to-end rehearse:
  - browser path
  - fallback path
  - comparison path

## 19. Final Demo Acceptance Criteria

The final product is ready only if:

- one startup prompt arms OpenClaw
- one business-event trigger starts the incident
- OpenClaw wakes without a second long prompt
- Stratus outputs a shortlist and guarded sequence
- the dangerous reflex is explicitly rejected
- step 1 executes successfully
- the system shows whether step 2 is needed
- final verdict includes baseline vs guardrail over time
- case writeback is compact and retrievable

## 20. Sources

- HL7 FHIR Slot: <https://hl7.org/fhir/r4/slot.html>
- HL7 FHIR Appointment: <https://hl7.org/fhir/r4/appointment.html>
- HL7 FHIR Schedule: <https://www.hl7.org/fhir/schedule.html>
- |                                                                                                                                                             |            |          |           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|----------|-----------|
| HHS                                                                                                                                                         | telehealth | workflow | planning: |
| <a href="https://telehealth.hhs.gov/providers/planning-your-telehealth-workflow">https://telehealth.hhs.gov/providers/planning-your-telehealth-workflow</a> |            |          |           |
- Prometheus histograms and quantiles: <https://prometheus.io/docs/practices/histograms/>
- Prometheus query functions: <https://prometheus.io/docs/prometheus/latest/querying/functions/>
- Alertmanager concepts: <https://prometheus.io/docs/alerting/latest/alertmanager/>
- SimPy basic concepts: [https://simpy.readthedocs.io/en/stable/simpy\\_intro/basic\\_concepts.html](https://simpy.readthedocs.io/en/stable/simpy_intro/basic_concepts.html)
- DuckDB Parquet overview: <https://duckdb.org/docs/current/data/parquet/overview.html>